Model Performance Comparison

Model	Test MAE (°C)
SimpleRNN	2.456
Stacked GRU	2.509
LSTM	2.540
Dense front-end LSTM	2.618
Naive baseline	2.622
Dense before/after LSTM	2.623

The objective of this experiment was to evaluate different neural network architectures for time-series forecasting using the Jena climate dataset. Model performance was measured using mean absolute error (MAE), with lower values indicating more accurate temperature predictions. Five neural network configurations were compared against a naive baseline.

To my frustration, the SimpleRNN achieved the best performance, with a test MAE of 2.456°C. The stacked GRU followed with 2.509°C, while the LSTM achieved 2.540°C. These results indicate that all three recurrent neural network architectures outperformed the naive baseline, which had a test MAE of 2.622°C. The SimpleRNN reduced prediction error by approximately 6.3% compared with the baseline, demonstrating that recurrent networks can capture useful temporal patterns in climate data.

The more complex architectures did not produce better results. The dense front-end LSTM achieved a test MAE of 2.618°C, offering only a marginal improvement over the baseline. The dense before/after LSTM performed slightly worse than the baseline, with an MAE of 2.623°C. These findings suggest that adding dense layers before (and before and after) recurrent layers does not necessarily improve forecasting accuracy and may introduce additional complexity without meaningful benefits.

Overall, the experiment demonstrated that model complexity alone does not guarantee improved predictive performance. The SimpleRNN provided the most accurate predictions among the evaluated models, while the stacked GRU and LSTM also demonstrated improvements over the baseline. Further experimentation with hyperparameters, regularization, and model architecture could potentially improve performance. These results highlight the importance of evaluating models empirically rather than assuming that more sophisticated architectures will yield better outcomes. Note** I attempted to recreate the stacked GRU layers with recurrent dropout model from the book; however the training time was over 13 hours and I lost my progress attempting it. I decided to proceed with my own versions of the model, though I acknowledge that going through with recurrent dropout would have likely improved performance.

In [3]:
```
!wget https://s3.amazonaws.com/keras-datasets/jena_climate_2009_2016.csv.zip
```

```
--2026-10-28 18:17:57--  https://s3.amazonaws.com/keras-datasets/jena_climate_2009_2
016.csv.zip
Resolving s3.amazonaws.com (s3.amazonaws.com)... 54.231.192.32, 16.15.172.103, 52.21
6.217.176, ...
Connecting to s3.amazonaws.com (s3.amazonaws.com)|54.231.192.32|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 13565642 (13M) [application/zip]
Saving to: ‘jena_climate_2009_2016.csv.zip’

jena_climate_2009_2 100%[===================>]  12.94M  48.8MB/s    in 0.3s

2026-10-28 18:17:58 (48.8 MB/s) - ‘jena_climate_2009_2016.csv.zip’ saved [13565642/1
3565642]
```

In [4]:
```
import zipfile

with zipfile.ZipFile("jena_climate_2009_2016.csv.zip", "r") as zip_ref:
    zip_ref.extractall(".")

print("Extraction complete.")
```

```
Extraction complete.
```

In [5]:
```
#Download and inspect data
import os
from tensorflow import keras
from tensorflow.keras import layers
fname = os.path.join("jena_climate_2009_2016.csv")

with open(fname) as f:
    data = f.read()

lines = data.splitlines()
header = lines[0].split(",")
lines = lines[1:]
```

```
2026-10-28 18:18:05.234773: I tensorflow/core/util/port.cc:153] oneDNN custom operat
ions are on. You may see slightly different numerical results due to floating-point
round-off errors from different computation orders. To turn them off, set the enviro
nment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-10-28 18:18:05.291635: I tensorflow/core/platform/cpu_feature_guard.cc:210] Thi
s TensorFlow binary is optimized to use available CPU instructions in performance-cr
itical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 FMA, in o
ther operations, rebuild TensorFlow with the appropriate compiler flags.
2026-10-28 18:18:07.471471: I tensorflow/core/util/port.cc:153] oneDNN custom operat
ions are on. You may see slightly different numerical results due to floating-point
round-off errors from different computation orders. To turn them off, set the enviro
nment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

In [6]:
```
# create numpy arrays for temp and raw data
import numpy as np
temperature = np.zeros((len(lines),))
```

```
raw_data = np.zeros((len(lines), len(header) - 1))
for i, line in enumerate(lines):
    values = [float(x) for x in line.split(",")[1:]]
    temperature[i] = values[1]
    raw_data[i, :] = values[:]

#integers for num train samples, val, and test
num_train_samples = int(0.5 * len(raw_data))
num_val_samples = int(0.25 * len(raw_data))
num_test_samples = len(raw_data) - num_train_samples - num_val_samples

#normalize raw data
mean = raw_data[:num_train_samples].mean(axis=0)
raw_data -= mean
std = raw_data[:num_train_samples].std(axis=0)
raw_data /= std
```

In [7]:

```
#Instantiate datasets with sampling rate, sequence length, delay (24 hours)
#and batch size
sampling_rate = 6
sequence_length = 120
delay = sampling_rate * (sequence_length + 24 - 1)
batch_size = 256

train_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[:-delay],
    targets=temperature[delay:],
    sampling_rate=sampling_rate,
    sequence_length=sequence_length,
    shuffle=True,
    batch_size=batch_size,
    start_index=0,
    end_index=num_train_samples)

val_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[:-delay],
    targets=temperature[delay:],
    sampling_rate=sampling_rate,
    sequence_length=sequence_length,
    shuffle=False,
    batch_size=batch_size,
    start_index=num_train_samples,
    end_index=num_train_samples + num_val_samples)

test_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[:-delay],
    targets=temperature[delay:],
    sampling_rate=sampling_rate,
    sequence_length=sequence_length,
    shuffle=False,
    batch_size=batch_size,
    start_index=num_train_samples + num_val_samples)
```

```
WARNING: All log messages before absl::InitializeLog() is called are written to STDE
RR
I0000 00:00:1793225893.569663    3460 gpu_device.cc:2020] Created device /job:localh
ost/replica:0/task:0/device:GPU:0 with 3539 MB memory:  -> device: 0, name: NVIDIA G
eForce RTX 4050 Laptop GPU, pci bus id: 0000:01:00.0, compute capability: 8.9
```

In [25]:
```python
#Naive Method…The common sense baseline
def evaluate_naive_method(dataset):
    total_abs_err = 0.
    samples_seen = 0
    for samples, targets in dataset:
        preds = samples[:, -1, 1] * std[1] + mean[1]
        total_abs_err += np.sum(np.abs(preds - targets))
        samples_seen += samples.shape[0]
    return total_abs_err / samples_seen

Naive_test=evaluate_naive_method(test_dataset)
```

```
2026-10-28 19:24:54.431926: I tensorflow/core/framework/local_rendezvous.cc:407] Loc
al rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
```

In [12]:
```python
from pathlib import Path

matches = list(Path.home().rglob("*SimpleRNN*"))

for file in matches:
    print(file)
```

```
/home/claxta11/ml/AML Assignment 3/SimpleRNN_Model.keras
```

In [13]:
```python
simple_rnn_model = keras.models.load_model(
    "/home/claxta11/ml/AML Assignment 3/SimpleRNN_Model.keras")
```

In [11]:
```python
#SimpleRNN Model
inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x= layers.SimpleRNN(16)(inputs)
outputs=layers.Dense(1)(x)
simple_rnn_model= keras.Model(inputs, outputs)

callbacks = [
    keras.callbacks.ModelCheckpoint(
        "SimpleRNN_Model.keras",
        monitor="val_mae",
        mode="min",
        save_best_only=True
    )
]
simple_rnn_model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history_SimpleRNN = simple_rnn_model.fit(train_dataset,
                    epochs=30,
                    validation_data=val_dataset,
                    callbacks=callbacks)
simple_rnn_model= keras.models.load_model("SimpleRNN_Model.keras")
```

```
Epoch 1/30
```

```
/home/claxta11/miniconda3/envs/tensorflow/lib/python3.12/site-packages/keras/src/trainers/epoch_iterator.py:74: UserWarning: `shuffle=True` was passed, but will be ignored since the data `x` was provided as a tf.data.Dataset. The Dataset is expected to already be shuffled (via `.shuffle(buffer_size)`).
  self.data_adapter = data_adapters.get_data_adapter(
2026-10-28 17:54:38.801488: I external/local_xla/xla/service/service.cc:163] XLA service 0x7120e00e77c0 initialized for platform CUDA (this does not guarantee that XLA will be used). Devices:
2026-10-28 17:54:38.801522: I external/local_xla/xla/service/service.cc:171]   StreamExecutor device (0): NVIDIA GeForce RTX 4050 Laptop GPU, Compute Capability 8.9
2026-10-28 17:54:38.868408: I tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_util.cc:269] disabling MLIR crash reproducer, set env var `MLIR_CRASH_REPRODUCER_DIRECTORY` to enable.
2026-10-28 17:54:39.226465: I external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:473] Loaded cuDNN version 92600
  3/819 ━━━━━━━━━━━━━━━━━━━━ 30s 37ms/step - loss: 153.0818 - mae: 10.0856
I0000 00:00:1793224480.459815     825 device_compiler.h:196] Compiled cluster using XLA!  This line is logged at most once for the lifetime of the process.
```

```
819/819 ━━━━━━━━━━━━━━━━━━━━ 41s 47ms/step - loss: 40.1938 - mae: 4.6434 - val_loss:
11.9516 - val_mae: 2.6201
Epoch 2/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 39s 48ms/step - loss: 11.1718 - mae: 2.6009 - val_loss:
9.4996 - val_mae: 2.3968
Epoch 3/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 38s 47ms/step - loss: 10.4838 - mae: 2.5259 - val_loss:
9.2860 - val_mae: 2.3711
Epoch 4/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 38s 46ms/step - loss: 10.3141 - mae: 2.5040 - val_loss:
9.1422 - val_mae: 2.3514
Epoch 5/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 39s 47ms/step - loss: 10.2120 - mae: 2.4901 - val_loss:
9.0556 - val_mae: 2.3381
Epoch 6/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 38s 46ms/step - loss: 10.1370 - mae: 2.4795 - val_loss:
8.9954 - val_mae: 2.3304
Epoch 7/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 38s 47ms/step - loss: 10.0706 - mae: 2.4707 - val_loss:
9.0240 - val_mae: 2.3308
Epoch 8/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 38s 47ms/step - loss: 10.0167 - mae: 2.4635 - val_loss:
9.1008 - val_mae: 2.3406
Epoch 9/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 38s 47ms/step - loss: 9.9609 - mae: 2.4553 - val_loss:
8.9088 - val_mae: 2.3146
Epoch 10/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 39s 47ms/step - loss: 9.8969 - mae: 2.4471 - val_loss:
9.0344 - val_mae: 2.3347
Epoch 11/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 39s 47ms/step - loss: 9.8428 - mae: 2.4402 - val_loss:
8.9640 - val_mae: 2.3291
Epoch 12/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 38s 47ms/step - loss: 9.7911 - mae: 2.4350 - val_loss:
9.0503 - val_mae: 2.3408
Epoch 13/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 38s 46ms/step - loss: 9.7581 - mae: 2.4315 - val_loss:
8.9993 - val_mae: 2.3373
Epoch 14/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 42s 47ms/step - loss: 9.7210 - mae: 2.4278 - val_loss:
8.8964 - val_mae: 2.3195
Epoch 15/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 39s 48ms/step - loss: 9.6754 - mae: 2.4231 - val_loss:
8.9141 - val_mae: 2.3216
Epoch 16/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 38s 47ms/step - loss: 9.6423 - mae: 2.4195 - val_loss:
8.9228 - val_mae: 2.3208
Epoch 17/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 38s 46ms/step - loss: 9.6080 - mae: 2.4143 - val_loss:
9.3093 - val_mae: 2.3735
Epoch 18/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 38s 46ms/step - loss: 9.5885 - mae: 2.4116 - val_loss:
8.8953 - val_mae: 2.3184
Epoch 19/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 38s 46ms/step - loss: 9.5633 - mae: 2.4078 - val_loss:
9.0780 - val_mae: 2.3418
```

```
Epoch 20/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 38s 47ms/step - loss: 9.5386 - mae: 2.4049 - val_loss:
8.9581 - val_mae: 2.3245
Epoch 21/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 39s 48ms/step - loss: 9.5189 - mae: 2.4028 - val_loss:
8.9500 - val_mae: 2.3226
Epoch 22/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 39s 48ms/step - loss: 9.5001 - mae: 2.3999 - val_loss:
9.0576 - val_mae: 2.3340
Epoch 23/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 39s 47ms/step - loss: 9.4730 - mae: 2.3973 - val_loss:
9.0973 - val_mae: 2.3382
Epoch 24/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 40s 48ms/step - loss: 9.4420 - mae: 2.3939 - val_loss:
9.2459 - val_mae: 2.3613
Epoch 25/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 40s 48ms/step - loss: 9.4162 - mae: 2.3905 - val_loss:
9.0128 - val_mae: 2.3285
Epoch 26/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 39s 47ms/step - loss: 9.3974 - mae: 2.3876 - val_loss:
9.1260 - val_mae: 2.3480
Epoch 27/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 36s 44ms/step - loss: 9.3728 - mae: 2.3835 - val_loss:
9.1793 - val_mae: 2.3501
Epoch 28/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 37s 44ms/step - loss: 9.3456 - mae: 2.3799 - val_loss:
9.0424 - val_mae: 2.3360
Epoch 29/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 36s 44ms/step - loss: 9.3226 - mae: 2.3776 - val_loss:
9.0885 - val_mae: 2.3433
Epoch 30/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 37s 44ms/step - loss: 9.3022 - mae: 2.3744 - val_loss:
9.0843 - val_mae: 2.3374
```

In [14]:

```
#LSTM Model
inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.LSTM(16)(inputs)
outputs = layers.Dense(1)(x)
lstm_model= keras.Model(inputs, outputs)

callbacks = [
    keras.callbacks.ModelCheckpoint("lstm.keras", monitor="val_mae",
    mode="min", save_best_only=True),
     keras.callbacks.EarlyStopping(
        monitor="val_mae",
        mode="min",
        patience=5,
        restore_best_weights=True
    )
]

lstm_model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history_LSTM = lstm_model.fit(train_dataset,
                    epochs=30,
                    validation_data=val_dataset,
```

```
                    callbacks=callbacks)
lstm_model= keras.models.load_model("lstm.keras")
```

```
Epoch 1/30
/home/claxta11/miniconda3/envs/tensorflow/lib/python3.12/site-packages/keras/src/trainers/epoch_iterator.py:74: UserWarning: `shuffle=True` was passed, but will be ignored since the data `x` was provided as a tf.data.Dataset. The Dataset is expected to already be shuffled (via `.shuffle(buffer_size)`).
  self.data_adapter = data_adapters.get_data_adapter(
2026-10-28 18:22:24.760793: I external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:473] Loaded cuDNN version 92600
819/819 ━━━━━━━━━━━━━━━━━━━━ 25s 29ms/step - loss: 41.9039 - mae: 4.7287 - val_loss: 12.5667 - val_mae: 2.6927
Epoch 2/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 25s 31ms/step - loss: 11.1129 - mae: 2.5850 - val_loss: 9.5489 - val_mae: 2.4006
Epoch 3/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 25s 31ms/step - loss: 9.7377 - mae: 2.4252 - val_loss: 9.6283 - val_mae: 2.4067
Epoch 4/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 26s 31ms/step - loss: 9.2642 - mae: 2.3660 - val_loss: 9.5957 - val_mae: 2.3977
Epoch 5/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 26s 32ms/step - loss: 8.8538 - mae: 2.3145 - val_loss: 9.8528 - val_mae: 2.4408
Epoch 6/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 26s 32ms/step - loss: 8.5555 - mae: 2.2747 - val_loss: 9.5687 - val_mae: 2.4112
Epoch 7/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 27s 33ms/step - loss: 8.2938 - mae: 2.2413 - val_loss: 9.6052 - val_mae: 2.4151
Epoch 8/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 26s 31ms/step - loss: 8.0965 - mae: 2.2168 - val_loss: 9.6866 - val_mae: 2.4253
Epoch 9/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 26s 32ms/step - loss: 7.9304 - mae: 2.1950 - val_loss: 10.1917 - val_mae: 2.4787
```

In [15]:
```
import os
print(os.path.exists("lstm.keras"))
```

```
True
```

In [16]:
```
lstm_model = keras.models.load_model("lstm.keras")
```

In [18]:
```
#LSTM With More Units, 2 Dense Top Layers Before, and Dropout
inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.Dropout(0.3)(inputs)
x=layers.Dense(16, activation="relu")(x)
x = layers.Dropout(0.3)(x)
x=layers.Dense(16, activation= "relu")(x)
x = layers.LSTM(32)(x)
outputs = layers.Dense(1)(x)
dense_frontend_lstm_model= keras.Model(inputs, outputs)

callbacks = [
```

```
    keras.callbacks.ModelCheckpoint("dense_frontend_lstm_model.keras", monitor="val
    mode="min", save_best_only=True),
     keras.callbacks.EarlyStopping(
        monitor="val_mae",
        mode="min",
        patience=5,
        restore_best_weights=True
    )

]
dense_frontend_lstm_model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history_Improved_LSTM = dense_frontend_lstm_model.fit(train_dataset,
                    epochs=30,
                    validation_data=val_dataset,
                    callbacks=callbacks)
dense_frontend_lstm_model= keras.models.load_model("dense_frontend_lstm_model.keras
```

```
Epoch 1/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 31s 37ms/step - loss: 24.3550 - mae: 3.5908 - val_loss:
10.3480 - val_mae: 2.4922
Epoch 2/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 32s 39ms/step - loss: 11.4748 - mae: 2.6550 - val_loss:
9.6877 - val_mae: 2.4371
Epoch 3/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 31s 37ms/step - loss: 10.5807 - mae: 2.5525 - val_loss:
11.1420 - val_mae: 2.6124
Epoch 4/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 31s 38ms/step - loss: 9.9416 - mae: 2.4789 - val_loss:
10.4456 - val_mae: 2.5252
Epoch 5/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 31s 38ms/step - loss: 9.4429 - mae: 2.4135 - val_loss:
11.5331 - val_mae: 2.6392
Epoch 6/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 31s 38ms/step - loss: 8.9417 - mae: 2.3495 - val_loss:
11.5819 - val_mae: 2.6449
Epoch 7/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 30s 37ms/step - loss: 8.6047 - mae: 2.3012 - val_loss:
11.7206 - val_mae: 2.6678
```

In [20]:
```
os.chdir("/home/claxta11/ml/AML Assignment 3")
print(os.getcwd())
```

```
/home/claxta11/ml/AML Assignment 3
```

In [21]:
```
#Stacked GRU Model with Dense at end
inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.GRU(32, return_sequences=True)(inputs)
x = layers.GRU(32)(x)
x = layers.Dropout(0.2)(x)
x=layers.Dense(16, activation="relu")(x)
outputs = layers.Dense(1)(x)
stacked_gru_model= keras.Model(inputs, outputs)

callbacks = [
    keras.callbacks.ModelCheckpoint("Stacked_gru_dropout.keras", monitor="val_mae",
    mode="min", save_best_only=True),
     keras.callbacks.EarlyStopping(
```

```
        monitor="val_mae",
        mode="min",
        patience=5,
        restore_best_weights=True
    )

]
stacked_gru_model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history_Stacked_GRU = stacked_gru_model.fit(train_dataset,
                    epochs=30,
                    validation_data=val_dataset,
                    callbacks=callbacks)
stacked_gru_model= keras.models.load_model("Stacked_gru_dropout.keras")
```

```
Epoch 1/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 40s 48ms/step - loss: 18.5905 - mae: 3.1341 - val_loss:
9.0542 - val_mae: 2.3511
Epoch 2/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 39s 47ms/step - loss: 10.3385 - mae: 2.5095 - val_loss:
9.4245 - val_mae: 2.3830
Epoch 3/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 39s 48ms/step - loss: 9.4871 - mae: 2.4113 - val_loss:
8.8502 - val_mae: 2.3099
Epoch 4/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 38s 47ms/step - loss: 8.7169 - mae: 2.3112 - val_loss:
9.2293 - val_mae: 2.3547
Epoch 5/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 39s 47ms/step - loss: 7.9504 - mae: 2.2053 - val_loss:
10.5669 - val_mae: 2.5248
Epoch 6/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 40s 48ms/step - loss: 7.1455 - mae: 2.0897 - val_loss:
11.3657 - val_mae: 2.6122
Epoch 7/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 39s 48ms/step - loss: 6.4135 - mae: 1.9783 - val_loss:
11.4476 - val_mae: 2.6149
Epoch 8/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 40s 48ms/step - loss: 5.7521 - mae: 1.8699 - val_loss:
11.4261 - val_mae: 2.6352
```

In [22]:
```
#Dense before and after LSTM
inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.Dense(32, activation="relu")(inputs)
x = layers.LSTM(32)(x)
x = layers.Dense(8, activation="relu")(x)
x = layers.Dropout(0.2)(x)
outputs = layers.Dense(1)(x)
dense_before_after_lstm_model= keras.Model(inputs, outputs)

callbacks = [
    keras.callbacks.ModelCheckpoint("Dense_Before_After_LSTM.keras", monitor="val_m
    mode="min", save_best_only=True),
     keras.callbacks.EarlyStopping(
        monitor="val_mae",
        mode="min",
        patience=5,
        restore_best_weights=True
```

```
    )

]
dense_before_after_lstm_model.compile(optimizer="rmsprop", loss="mse", metrics=["ma
history_Dense_Before_After_LSTM = dense_before_after_lstm_model.fit(train_dataset,
                    epochs=30,
                    validation_data=val_dataset,
                    callbacks=callbacks)
dense_before_after_lstm_model=keras.models.load_model("Dense_Before_After_LSTM.kera
```

```
Epoch 1/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 29s 35ms/step - loss: 33.2785 - mae: 4.1539 - val_loss:
10.0877 - val_mae: 2.4701
Epoch 2/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 27s 33ms/step - loss: 18.8868 - mae: 3.2516 - val_loss:
9.8991 - val_mae: 2.4496
Epoch 3/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 29s 35ms/step - loss: 16.7658 - mae: 3.0495 - val_loss:
9.8333 - val_mae: 2.4493
Epoch 4/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 28s 34ms/step - loss: 15.0210 - mae: 2.8696 - val_loss:
10.4565 - val_mae: 2.5373
Epoch 5/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 28s 34ms/step - loss: 13.7772 - mae: 2.7377 - val_loss:
11.1700 - val_mae: 2.6039
Epoch 6/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 28s 34ms/step - loss: 12.7082 - mae: 2.6202 - val_loss:
11.2891 - val_mae: 2.6229
Epoch 7/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 28s 35ms/step - loss: 11.8195 - mae: 2.5267 - val_loss:
11.9247 - val_mae: 2.7193
Epoch 8/30
819/819 ━━━━━━━━━━━━━━━━━━━━ 28s 35ms/step - loss: 11.0542 - mae: 2.4404 - val_loss:
11.4932 - val_mae: 2.6668
```

In [27]:
```
#Evaluating every version and printing as a table
import pandas as pd

summary_df = pd.DataFrame([
    {
        "Model": "Naive baseline",
        "Test MAE": Naive_test
    },
    {
        "Model": "SimpleRNN",
        "Test MAE": simple_rnn_model.evaluate(
            test_dataset,
            verbose=0
        )[1]
    },
    {
        "Model": "LSTM",
        "Test MAE": lstm_model.evaluate(
            test_dataset,
            verbose=0
        )[1]
```

```
    },
    {
        "Model": "Dense front-end LSTM",
        "Test MAE": dense_frontend_lstm_model.evaluate(
            test_dataset,
            verbose=0
        )[1]
    },
    {
        "Model": "Stacked GRU",
        "Test MAE": stacked_gru_model.evaluate(
            test_dataset,
            verbose=0
        )[1]
    },
    {
        "Model": "Dense before/after LSTM",
        "Test MAE": dense_before_after_lstm_model.evaluate(
            test_dataset,
            verbose=0
        )[1]
    }
])

summary_df = summary_df.sort_values(
    by="Test MAE",
    ascending=True
).reset_index(drop=True)

summary_df

#print to markdown format for copy pasting to top
print(summary_df.to_string(index=False, float_format=lambda x: f"{x:.3f}"))
```

```
                Model  Test MAE
           SimpleRNN     2.456
          Stacked GRU     2.509
                LSTM     2.540
  Dense front-end LSTM     2.618
        Naive baseline     2.622
Dense before/after LSTM     2.623
```

In [ ]: